

Data Science

Team 04_Data Lab

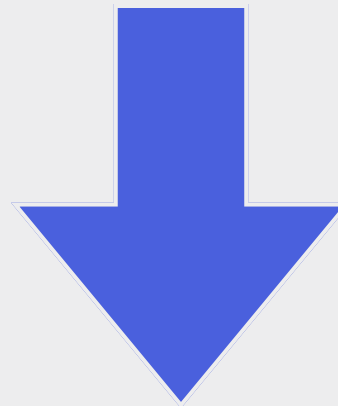
C389002 강하민
C389030 백세연
C384042 정재영

*** Contents**

- 1. Problem Definition**
- 2. Dataset Exploration**
- 3. Dataset Processing and Sampling**
- 4. Model Training**
- 5. Model Performance Comparison**
- 6. Conclusion**

* Problem Definition

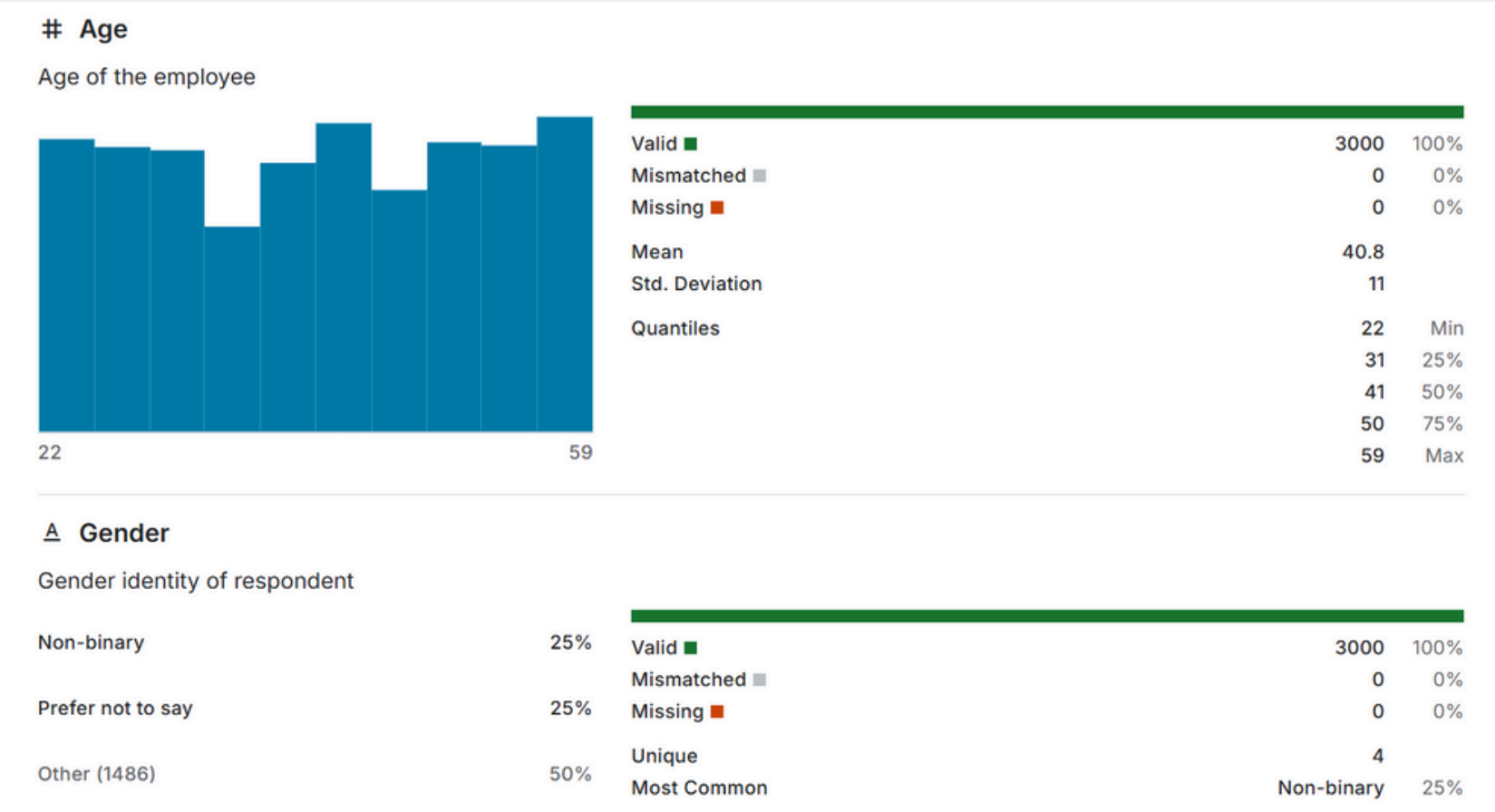
In order to proactively address the issues of short-term fatigue and decreased individual concentration, we predict the level of risk for concentration decline in real time



We predict the individual user's risk level of short-term fatigue and concentration decline by utilizing life log big data, such as sleep time, frequency of alcohol consumption, and BMI index.

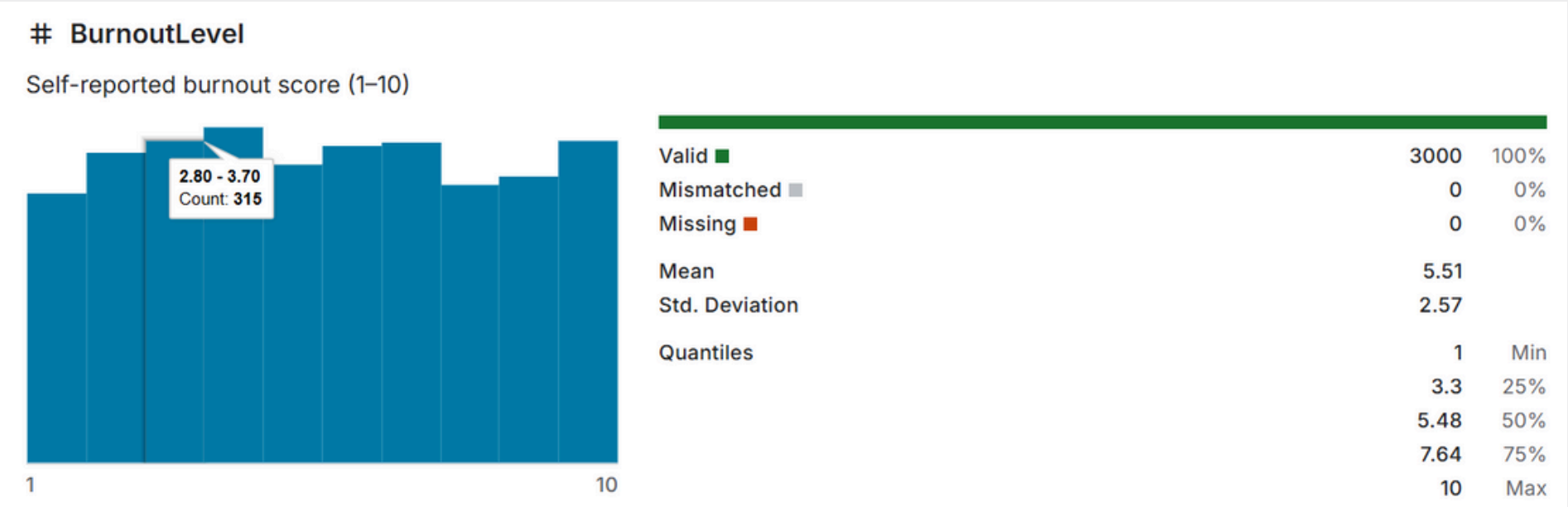
* Dataset Exploration

Data on **age**, **gender**, and **burnout level** were taken from kaggle.



Mental Health & Burnout in the Workplace

Survey of stress, burnout, and productivity



Sources: "<https://www.kaggle.com/datasets/khushikyad001/mental-health-and-burnout-in-the-workplace>"

* Dataset Exploration

Data on **physical activity** were taken from UC Irvine Machine Learning Repository

이름	유형
1_M	파일 폴더
2_F	파일 폴더
3_M	파일 폴더
4_F	파일 폴더
5_F	파일 폴더
6_M	파일 폴더
7_F	파일 폴더
8_M	파일 폴더
9_M	파일 폴더
10_M	파일 폴더
11_F	파일 폴더
12_M	파일 폴더
13_F	파일 폴더
14_F	파일 폴더
15_F	파일 폴더

 **UbiqLog (smartphone lifelogging)**
Donated on 6/15/2016

UbiqLog is the smartphone lifelogging tool that runs on the smartphone of 35 users for about 2 months.

Dataset Characteristics	Subject Area	Associated Tasks
Multivariate	Computer Science	Causal-Discovery
Feature Type	# Instances	# Features
-	9782222	-

Dataset Files	
File	Size
UbiqLog4UCI/35_F/log_6-11-2015.txt	9.8 MB
UbiqLog4UCI/35_F/log_6-2-2015.txt	9.7 MB
UbiqLog4UCI/1_M/log_11-12-2014.txt	9.4 MB
UbiqLog4UCI/35_F/log_6-1-2015.txt	9.3 MB

Dataset Information

Additional Information

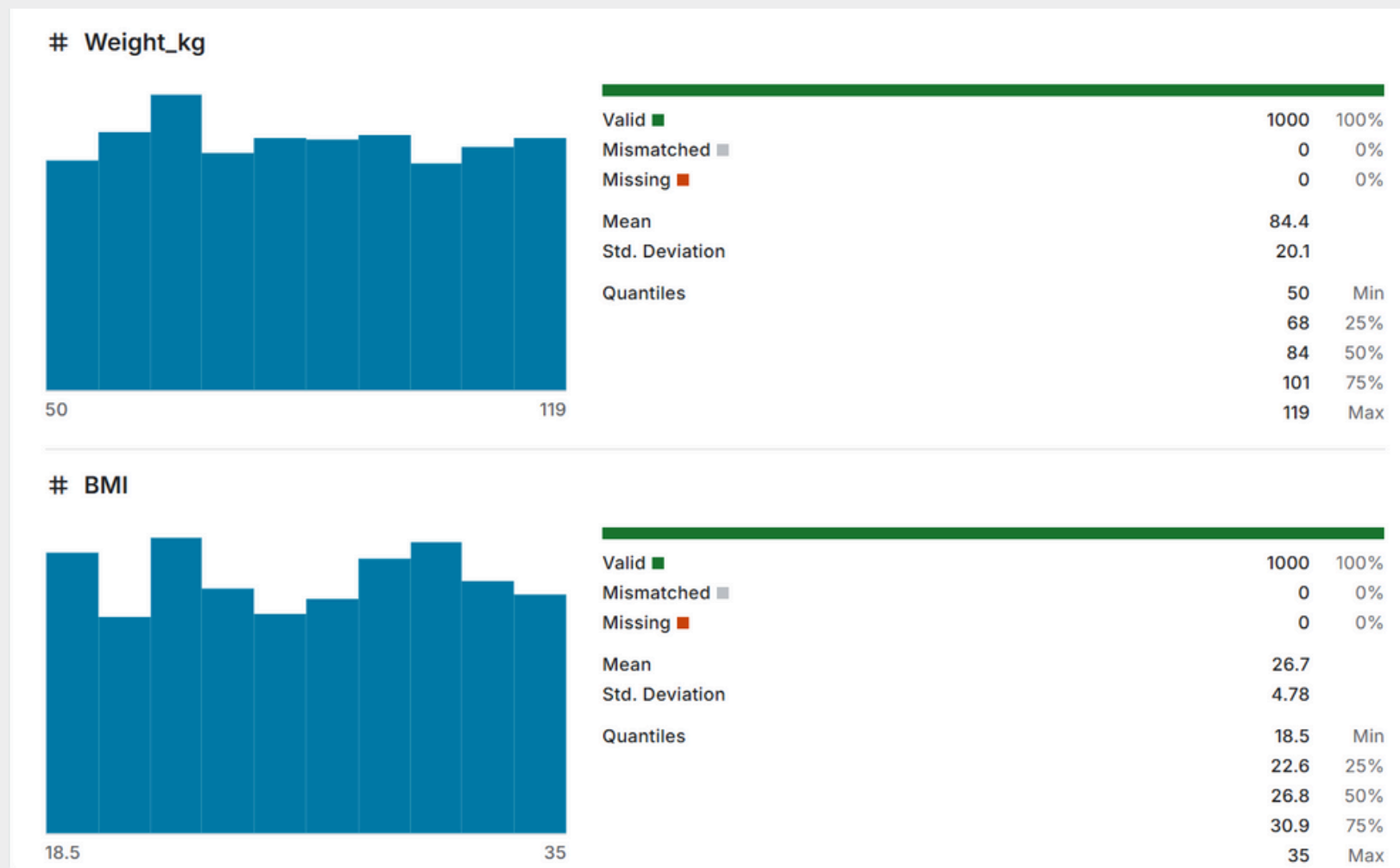
This is the first smartphone based lifelogging dataset that is going to be available for user of this dataset are obliged NOT to perform any sort of analysis that can harm the is not for any privacy related analysis that can re-identify users.

The UbiqLog tool is open source and accessible here: <https://github.com/Rezar/UbiqLog>

Sources: "<https://archive.ics.uci.edu/dataset/369/ubiqlog+smartphone+lifelogging>"

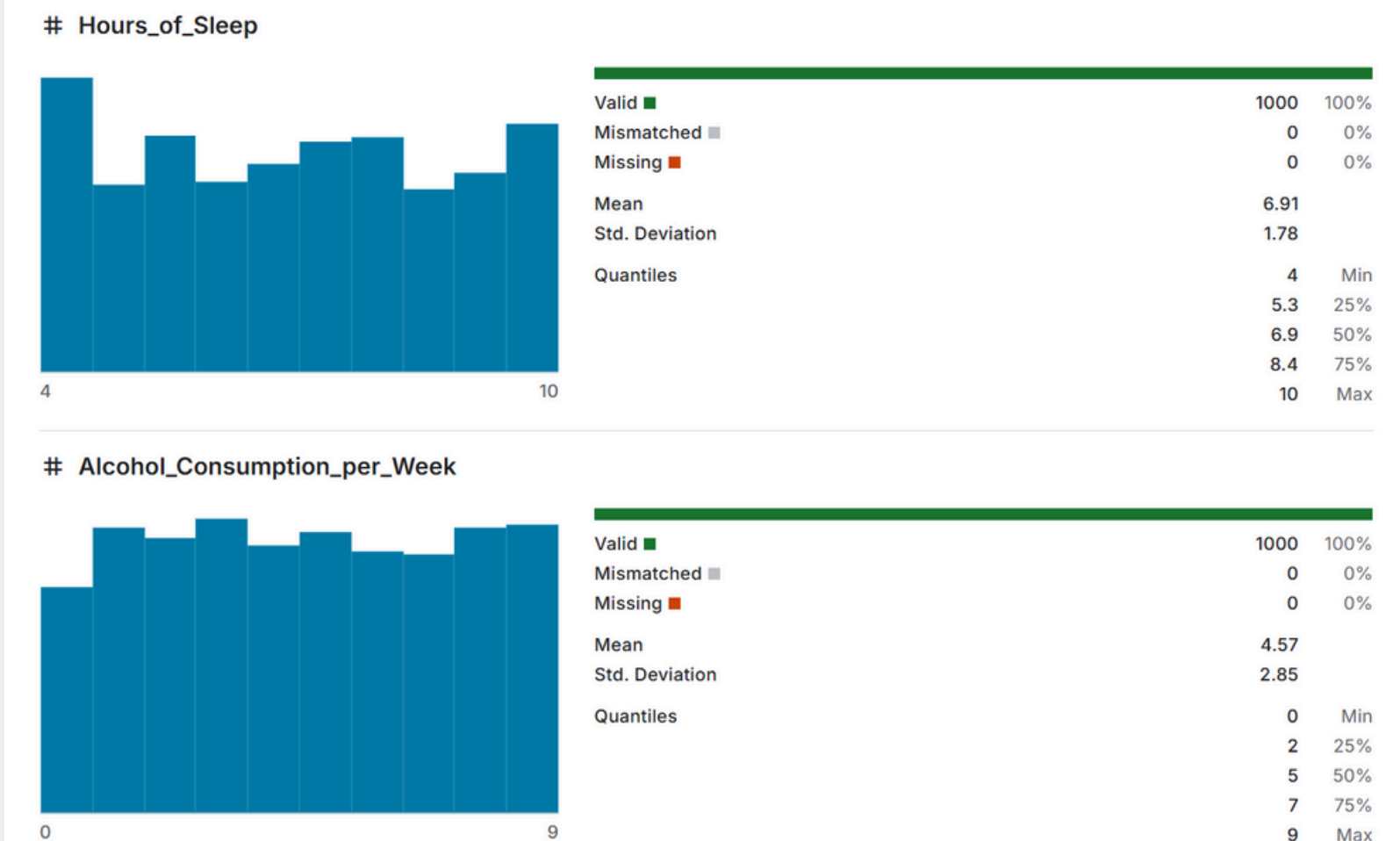
* Dataset Exploration

Data on **weight, BMI, sleep hours, Alchol consumption** were taken from kaggle.



Health & Lifestyle Dataset

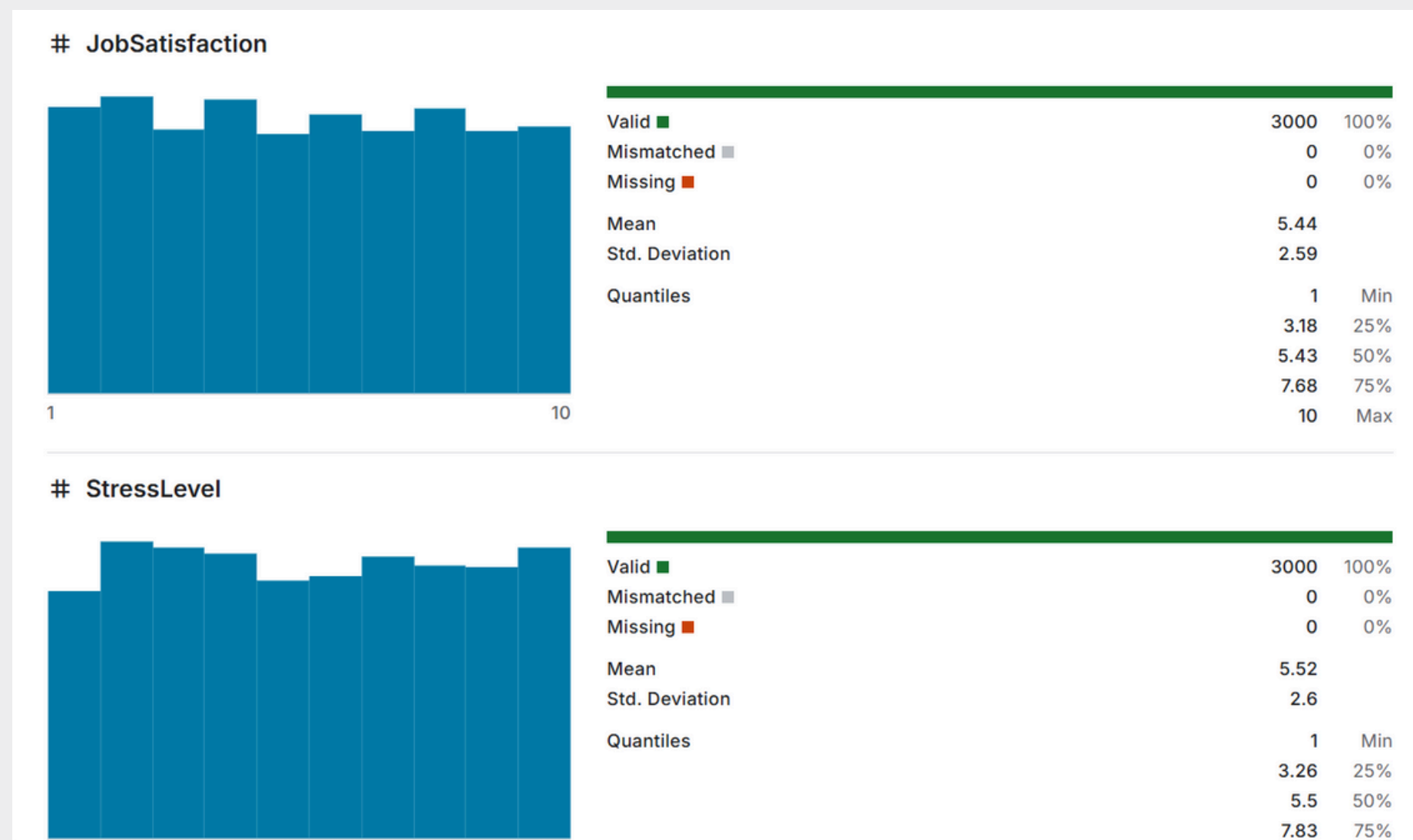
Health & Lifestyle Dataset – Analyze Well-being, Habits, and Trends Easily



Sources: "<https://www.kaggle.com/datasets/mahdimashayekhi/health-and-lifestyle-dataset?resource=download>"

* Dataset Exploration

Data on **stress level**, **job satisfaction** were taken from UC Irvine Machine Learning Repository



Health and Fitness Tracking 365 Days for 1000 User

Fitness tracking goals

Sources: "<https://www.kaggle.com/datasets/waqasishtiaq/fitness>"

* Dataset Preprocessing and Sampling

Preprocessing

- Combining data into one file

1	age	gender	physical_activity_hrs_pweek	weight_kg	drink_pweek	quality_of_sleep	sleephrs	stress_level	job_satisfaction	burnout_level	productivity_score
2	56	F	7.9	57.97258235	2	6	7	2.47	5.06	3.37	4.16
3	42	M	4	59.7642759	7	6	4.2	7.65	2	7.39	6.74
4	23	F	9.7	64.86787431	3	8	6.2	6.7	7.17	7.1	8.8
5	32	M	5.8	101.7149757	5	4	8.7	6.4	3.76	4.18	4.69
6	56	F	3.3	74.4314093	8	2	4.2	9.41	2.34	8.28	2.12
7	21	F	3.2	59.19656369	9	3	8.5	7.45	4.81	7.12	1.09
8	32	F	3.8	64.4279275	2	8	5.9	5.98	9.68	5.15	8.5
9	27	F	6.1	46.0850726	0	7	7.6	2.48	4.7	5.25	7.43
10	30	M	4.9	85.1116596	1	7	7.3	6.78	4.13	4.07	4.76
11	22	F	6.5	70.00750098	1	3	4.8	4	5	7.59	2.14
12	40	F	0.4	68.26519647	6	4	5.6	8.09	1.11	7.27	2.34
13	26	M	1.1	66.78738194	3	4	6.6	9.83	2.2	7.85	3.31
14	26	M	9.8	79.77769639	1	6	8.2	5.44	8.65	7.59	7.34
15	30	M	10	72.01444746	4	9	8.8	3.45	9.8	6.43	8.7
16	35	F	7.5	53.01575867	8	6	4.1	5.94	2.42	4.35	6.01
17	27	M	2.6	75.07035222	1	6	5.8	6.45	5.82	7.05	3.49
18	42	M	9.5	85.53586964	3	5	7.3	7.41	9.7	6.32	6.56
19	24	F	5.7	75.24654281	0	6	6.6	7.89	2.41	8.35	6.83
20	33	M	3.6	76.17172491	3	5	8.7	7.29	2.68	5.7	3.2
21	35	M	1.2	67.55160358	3	7	4.1	5.74	9.34	2.91	4.69
22	50	F	6.5	55.94371917	7	7	6.2	7.05	7.35	5.75	5.84
23	26	F	7.6	76.52260746	3	7	7.3	8.52	2.37	7.92	3.41
24	29	F	4.2	51.19913243	3	7	5.1	2.66	4.54	5.23	3.58
25	26	M	2.1	68.01151484	1	4	1.6	6.52	1.51	5.77	1.41

Rows:

2501 Rows
(2024-09-06~2025-09-05)

Columns:

11 Columns

* Dataset Preprocessing and Sampling

Preprocessing

- Missing Value Check and Handling

```
# 최종 결측치 개수 확인
# 모든 컬럼에 대해 결측치 (NA) 개수의 합계를 계산
final_missing_count <- colSums(is.na(df))

cat("--- 최종 결측치 개수 (모두 0이어야 함) ---\n")
print(final_missing_count)
```

→ No Missing Values Detected

```
--- 최종 결측치 개수 (모두 0이어야 함) ---
```

```
> print(final_missing_count)
```

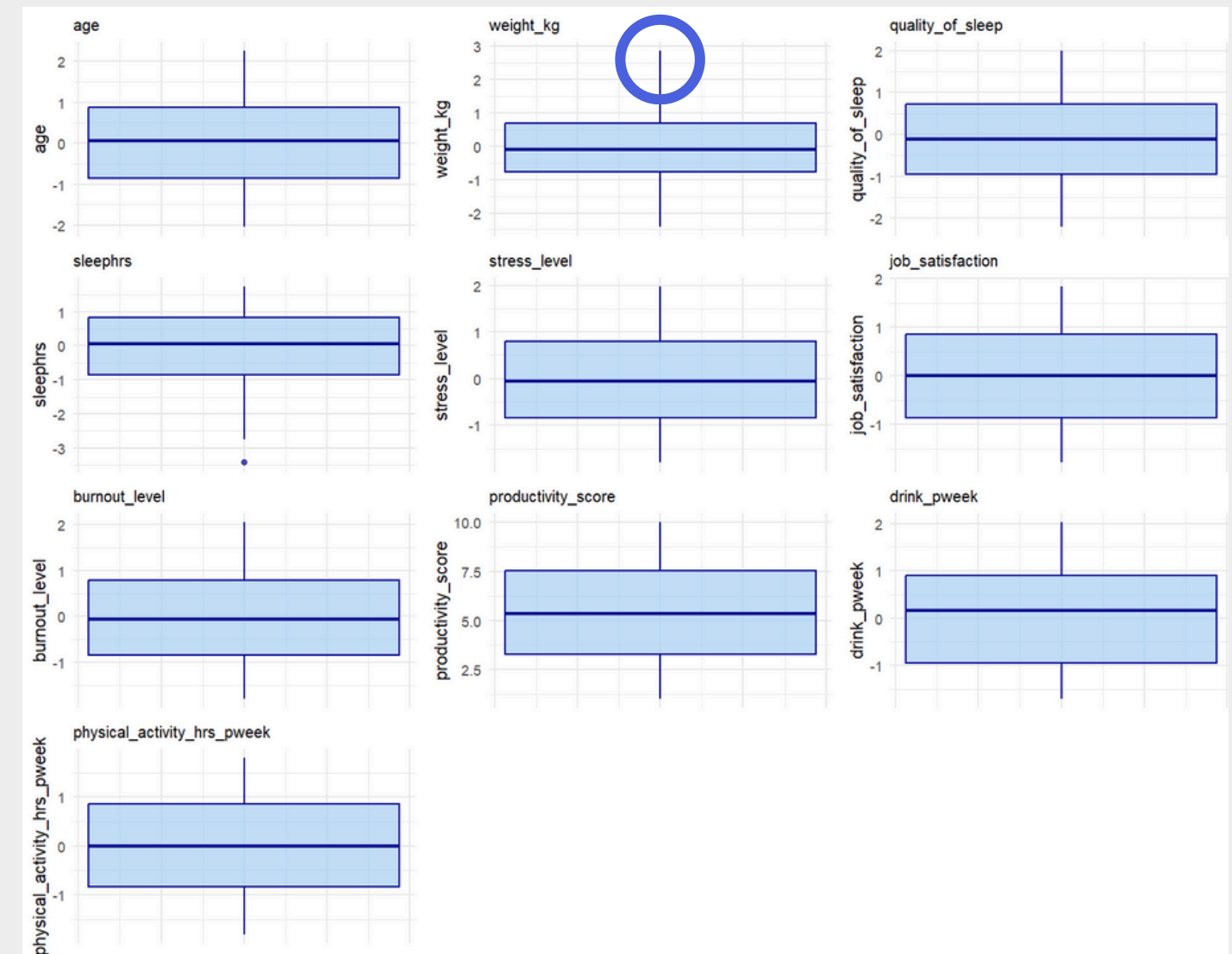
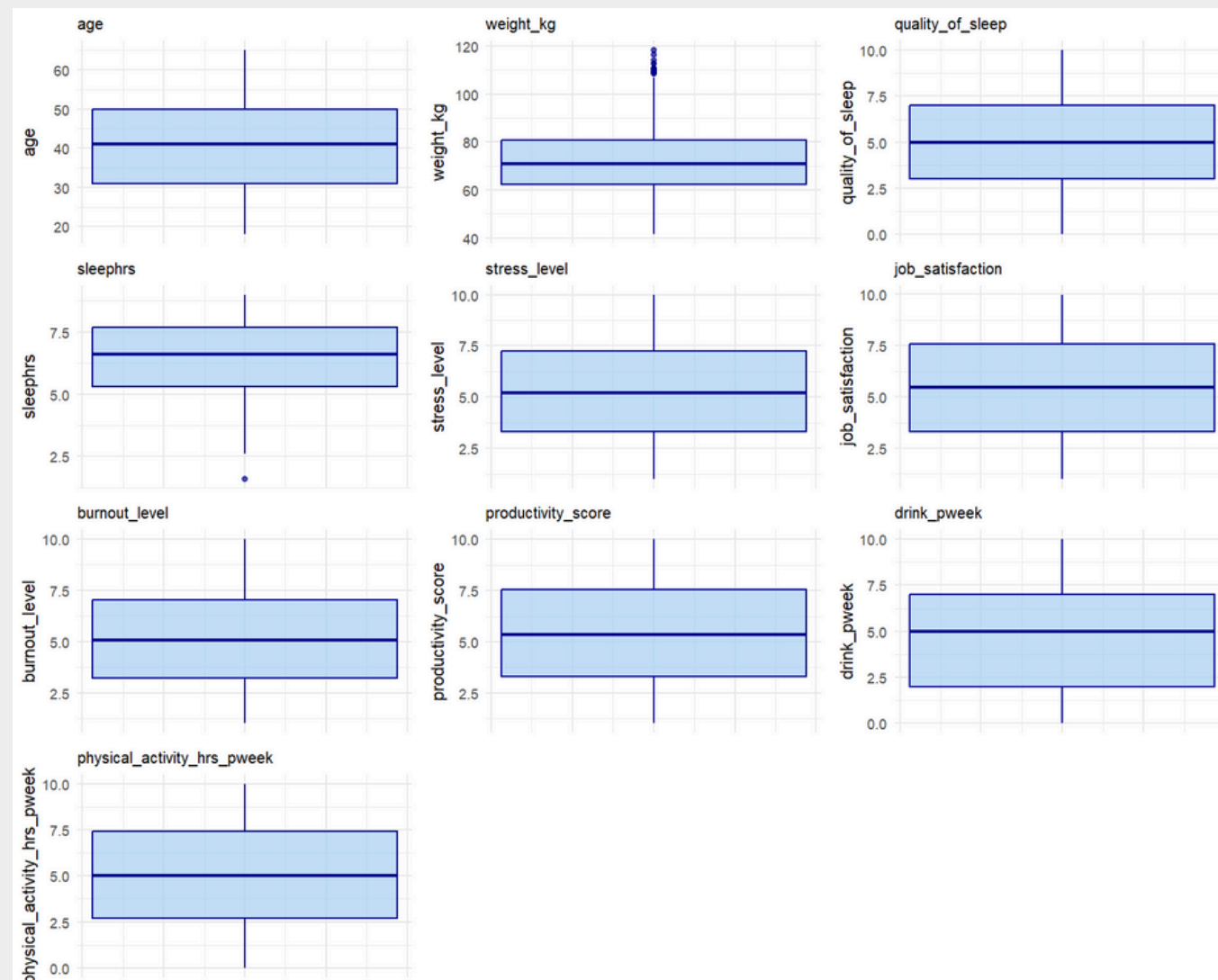
age	gender	physical_activity_hrs_pweek	weight_kg	drink_pweek
0	0	0	0	0
quality_of_sleep	sleephrs	stress_level	job_satisfaction	burnout_level
0	0	0	0	0
productivity_score				
0				

```
> |
```

* Dataset Preprocessing and Sampling

Preprocessing

- Outlier Detection

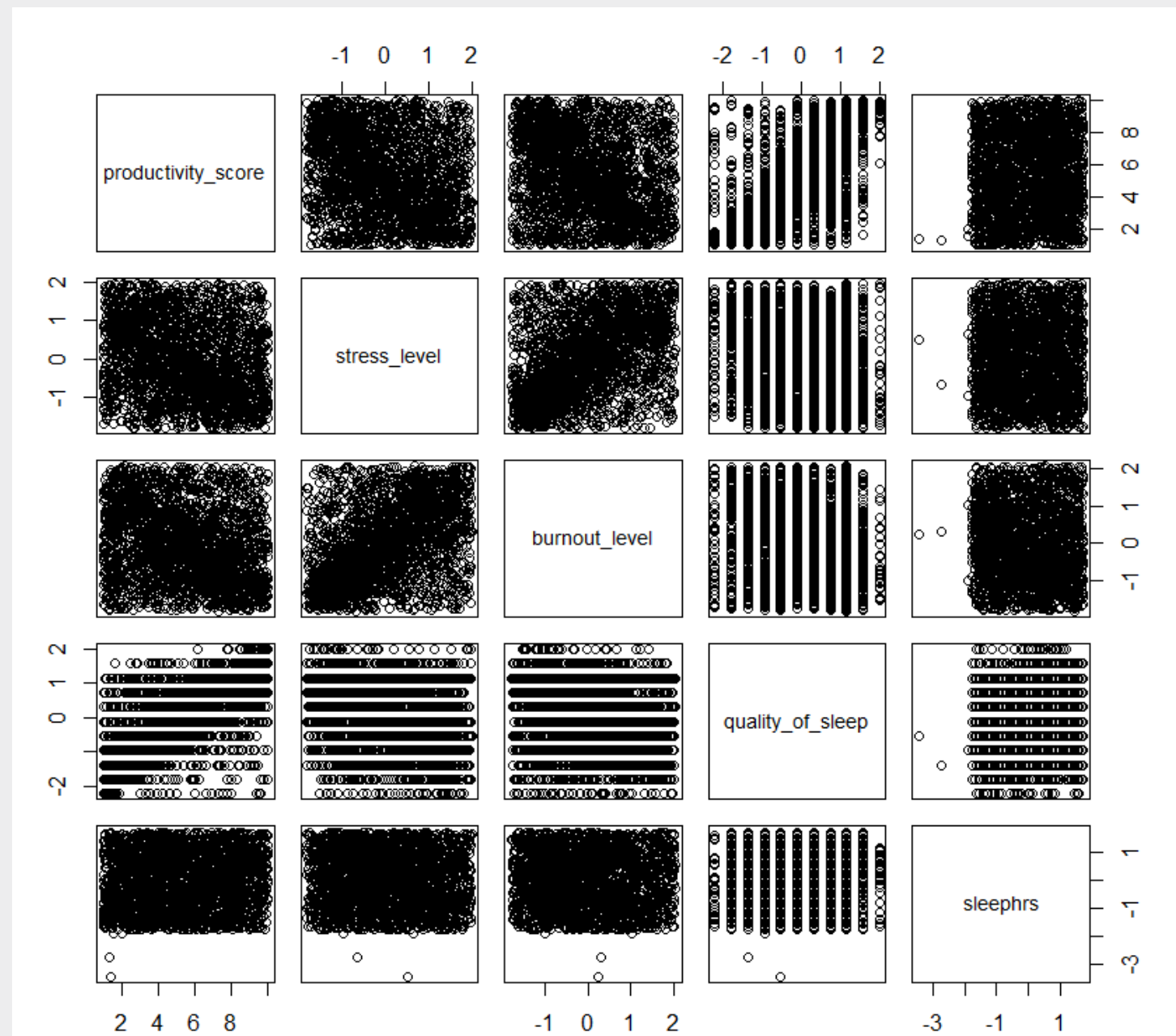


Upper outliers (extreme values) were removed because they can skew statistical models, leading to a decrease in reliability

* Dataset Preprocessing and Sampling

Preprocessing

- Dataset's Scatter Plot



The scatterplot between **stress_level** and **burnout_level** has a strong correlation with the dots spread together.

In the scatterplot of **quality_of_sleep** and **sleephrs**, discrete patterns of the shape of tight vertical lines or horizontal lines are prominent.

* Dataset Preprocessing and Sampling

Sampling

train_data

2002 rows
11 columns

test_data

499 rows
11 columns

To compare the predictive performance of the model, **20%** is selected as test data and **80%** is selected as a train dataset

* Dataset Preprocessing and Sampling

Final Dataset

V1: age	V6: sleephrs
V2: gender	V7: stress_level
V3: weight_kg	V8: job_satisfaction
V4: drink_pweek	V9: burnout_level
V5: quality_of_sleep	V10: productivity_score

Predict dependent variable **V10** using 9 independent variables

* Model Training

Linear Regression

```
def evaluate_models(X, y):  
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)  
  
    model_type = {  
        "LinearRegression": LinearRegression()  
    }  
  
    results = {}  
  
    for name, model in model_type.items():  
        model.fit(X_train, y_train)  
        preds = model.predict(X_test)  
  
        rmse = np.sqrt(mean_squared_error(y_test, preds))  
        r2 = r2_score(y_test, preds)  
  
        results[name] = {"RMSE": rmse, "R2": r2}  
  
    return pd.DataFrame(results).T
```

Data are separated into training (80%) and test (20%) and then predicted.

The model is evaluated with the RMSE and R^2 indices, and the performance is summarized in a table and returned.

* Model Training

Linear Regression

```
X_productivity = df_clean.drop(columns=["productivity_score"])
y_productivity = df_clean["productivity_score"]

result_productivity = evaluate_models(X_productivity, y_productivity)
print("\n=== 생산성 모델 성능 ===")
print(result_productivity)
```

```
=== 생산성 모델 성능 ===
              R2      RMSE
LinearRegression 0.419784 1.86011
```

The r^2 value is about 42% and the relationship between the variable and the productivity score is clear, and the prediction error is about rmse 1.9.

* Model Training

Linear Regression

```
from sklearn.inspection import permutation_importance
def get_feature_importances(models, X, y):
    results = {}

    for name, model in models.items():
        model.fit(X, y)

        if name == "LinearRegression":
            importance = model.coef_

        results[name] = pd.DataFrame({
            "feature": X.columns,
            "importance": importance
        }).sort_values("importance", ascending=False)

    return results
```

```
models = {
    "LinearRegression": LinearRegression()
}

feature_results_burnout = get_feature_importances(models, X_burnout, y_burnout)
feature_results_productivity = get_feature_importances(models, X_productivity, y_productivity)
```

Returns how much each variable (X) affects the target (y) with the coef association coefficient.

The coefficient of each variable is the magnitude and direction of the effect (+/-), and the greater the absolute value, the greater the effect.

* Model Training

Linear Regression

```
print("생산성 영향 변수 중요도\n")
for model_name, df in feature_results_productivity.items():
    print(f"=== {model_name} ===")
    for i, row in df.iterrows():
        print(f"{row['feature']:25} {row['importance']:.6f}")
    print("\n")
```

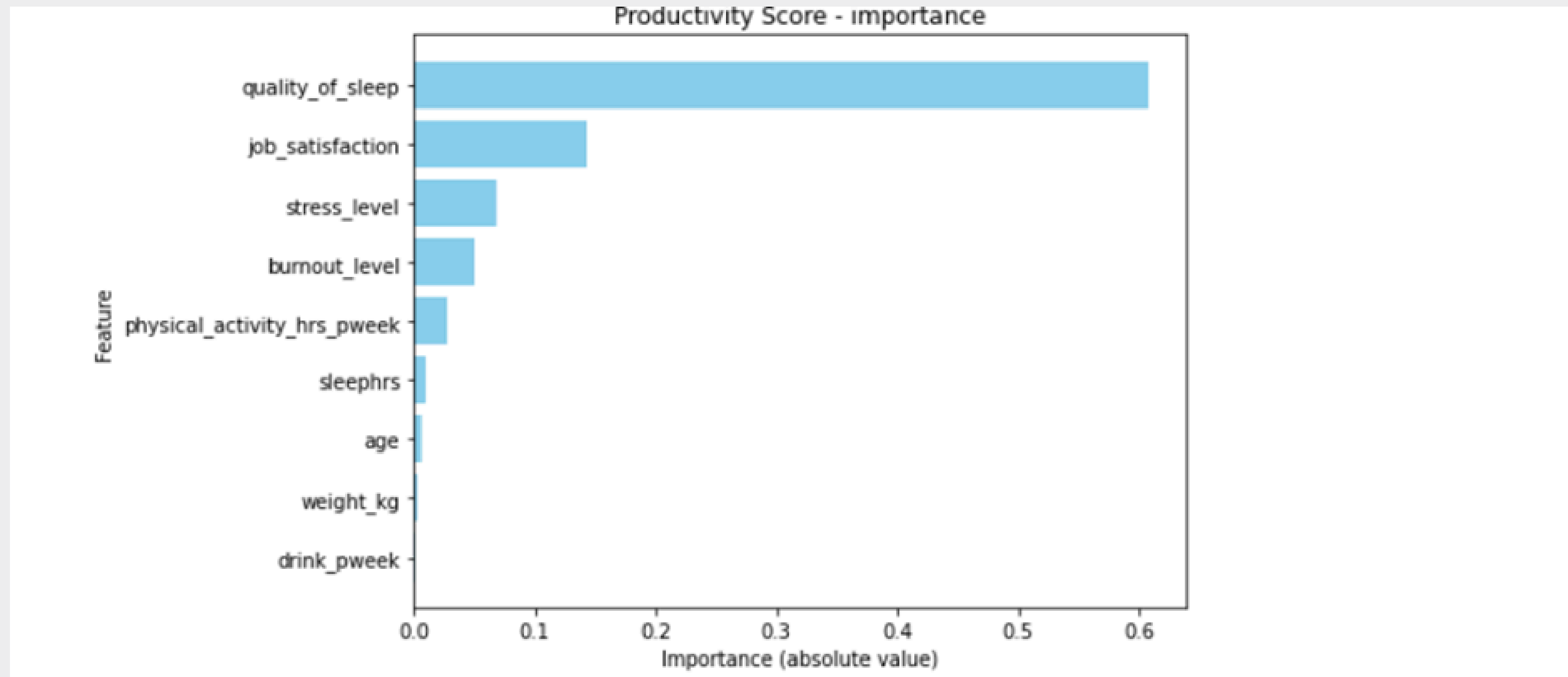
생산성 영향 변수 중요도

```
=== LinearRegression ===
quality_of_sleep      0.607698
job_satisfaction      0.142429
physical_activity_hrs_pweek 0.027749
age                   0.005248
weight_kg             0.003496
drink_pweek           -0.001665
sleephrs              -0.009754
burnout_level         -0.049769
stress_level          -0.067204
```

The variables that affect the productivity score are sleep quality, job satisfaction, stress index, and burnout level.

* Model Training

Linear Regression_Importance



Sort in ascending order from low-related to high-related variables.

As a result, productivity scores are highly associated with sleep quality and job satisfaction.

* Model Training

Random Forest

```
> library(randomForest)
> library(caret)
>
> # 교차 검증
> set.seed(42)
> tr_control <- trainControl(method = "cv",
+                             number = 10)
>
> cv_results_df <- data.frame(ntree = integer(),
+                             Mean_RMSE = numeric(),
+                             stringsAsFactors = FALSE)
>
> # ntree를 50부터 500까지 50씩 증가시키며 반복
> for (n in seq(from = 50, to = 500, by = 50)) {
+   cat(paste0(" 10-Fold CV 모델 학습 시작: ntree = ", n, "\n"))
+   rf_model_cv <- train(productivity_score ~ .,
+                         data = df_train,
+                         method = "rf",
+                         trControl = tr_control,
+                         tuneGrid = data.frame(mtry = 4), # mtry=4로 고정
+                         ntree = n) # ntree를 현재 반복 횟수로 지정
+   mean_rmse <- rf_model_cv$results$RMSE
+   cv_results_df[nrow(cv_results_df) + 1, ] <- c(n, mean_rmse)
+   cat(paste0("tree = ", n, " 모델 학습 완료 (10-Fold CV Mean RMSE: ", round(mean_rmse, 4), ")\n"))
+ }
```

10-fold cross validation

ntree=50,100,150,200,250,300,
350,400,450,500

mtry=4

* Model Training

Random Forest

```
ntree별 최종 Random Forest OOB MSE 비교
> print(results_df)
  ntree  OOB_MSE
1    50  3.753605
2   100  3.638814
3   150  3.591121
4   200  3.593898
5   250  3.552395
6   300  3.554567
7   350  3.536099
8   400  3.554702
9   450  3.525786
10  500  3.523329
```

Var explained = 43.43%
Best parameters ntree = 500
Validation_MSE = 3.523329

Random Forest 모델 학습 시작: ntree = 500

Call:

```
randomForest(formula = productivity_score ~ ., data = df_train, ntree = n, mtry = 4, importance = TRUE)
```

Type of random forest: regression

Number of trees: 500

No. of variables tried at each split: 4

Mean of squared residuals: 3.523329

% Var explained: 43.43

tree = 500 모델 학습 및 출력 완료 (최종 OOB MSE: 3.5233)

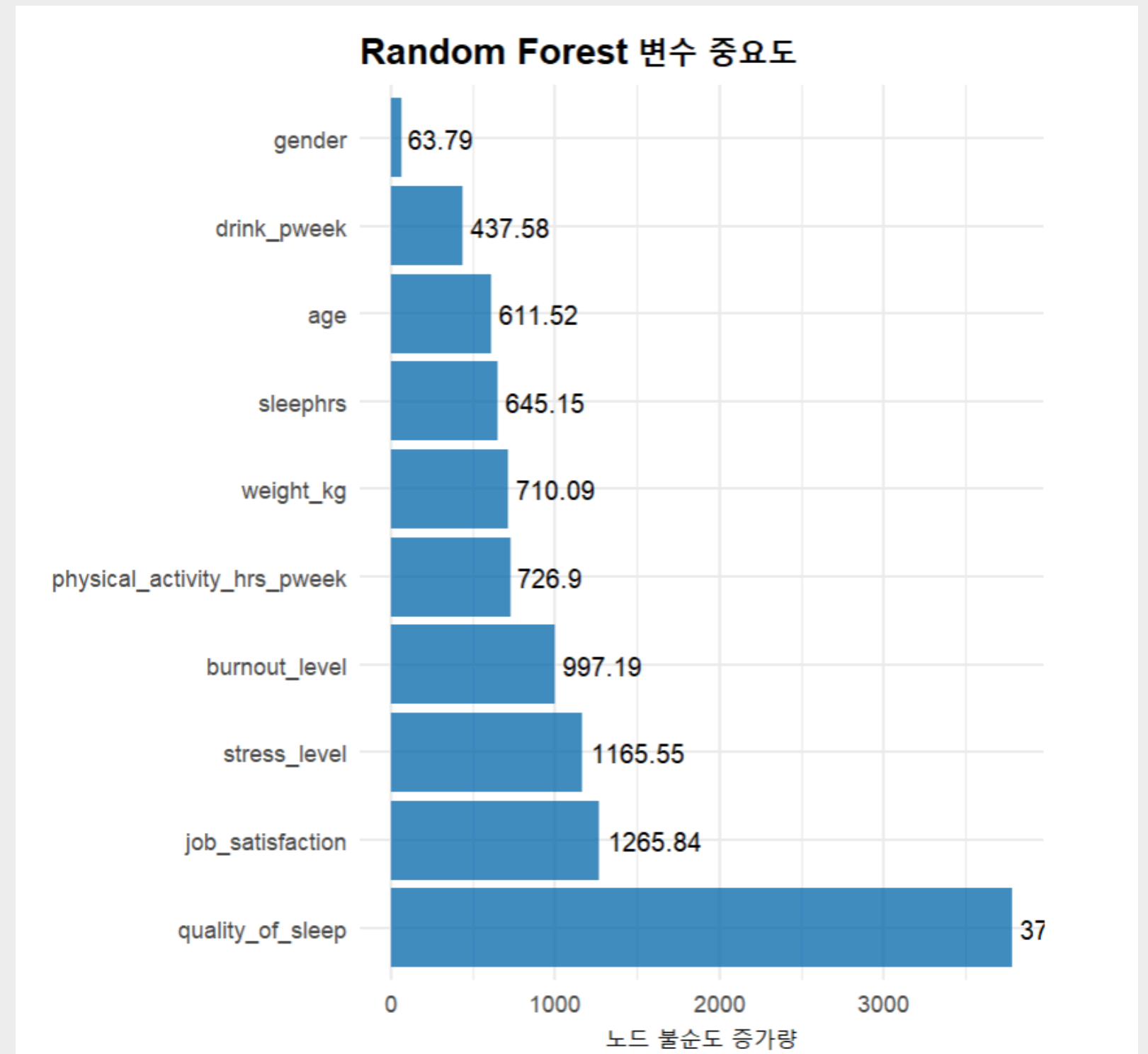
* Model Training

Random Forest_Importance

```
> print(importance_df)
```

	X.IncMSE	IncNodePurity
quality_of_sleep	37.2493771	4554.30422
job_satisfaction	6.3633684	1519.17557
burnout_level	6.7231591	1247.02373
stress_level	7.8906757	1168.35851
weight_kg	1.7759359	766.18651
physical_activity_hrs_pweek	-1.6741724	740.18720
sleephrs	3.0348341	718.56062
age	-0.4954406	667.63905
drink_pweek	0.6810071	481.71274
gender.M	1.0495618	68.29892
gender.F	1.6062065	58.19309

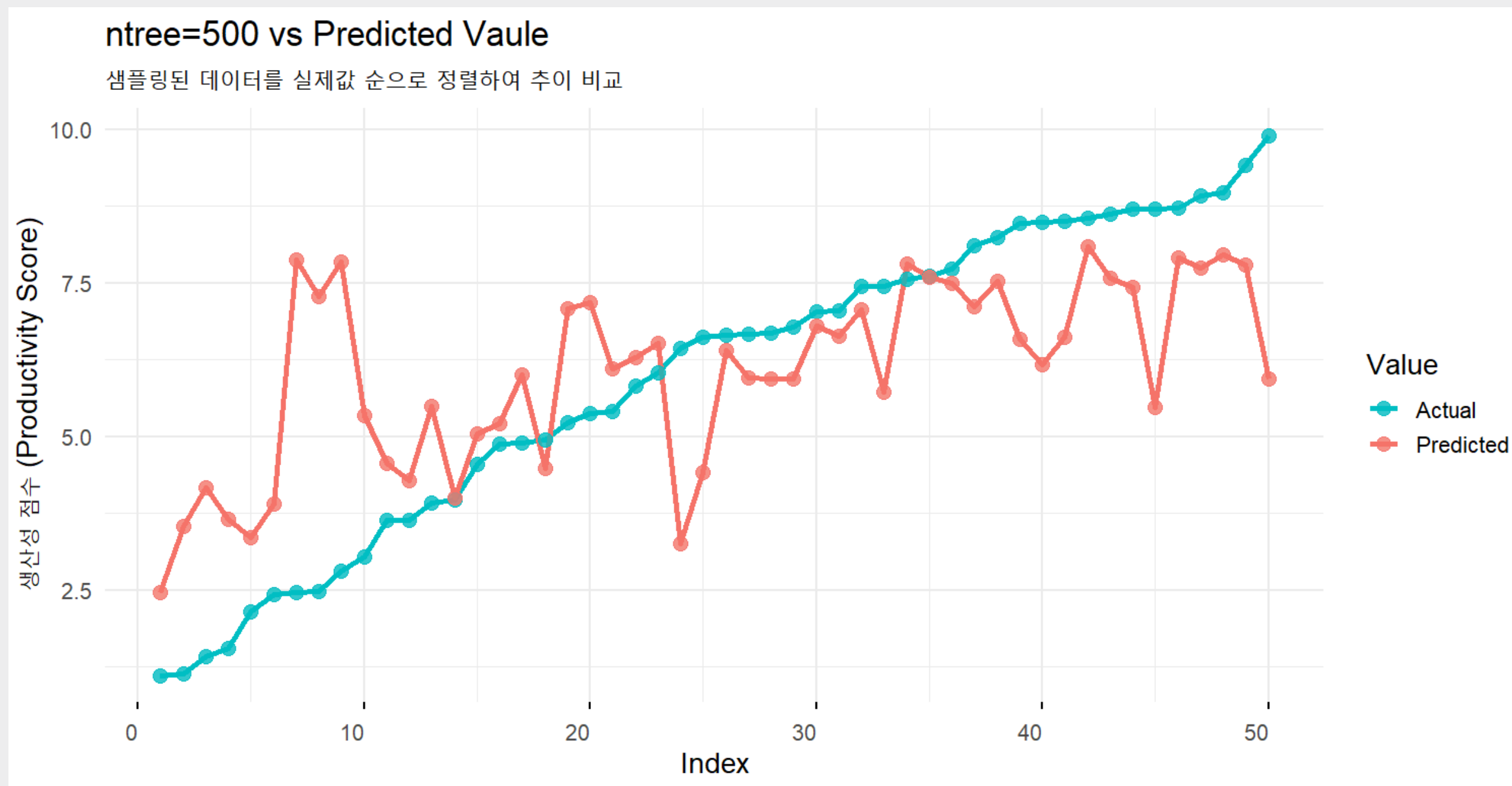
A negative physical_activity_hrs_pweek, age, means that the model's error (RMSE) decreased when the data of the variable were randomly mixed.



* Model Training

Random Forest_Result

```
> cat(sprintf("RMSE: %.4f\n", rmse_rf))  
RMSE: 1.8005
```



The overall flow of real-world values has been identified, but the large local error is regrettable for the performance of the predictive model, and the addition of variables is necessary to predict the productivity score.

* Model Training

KNN

Best K: 36 with CV Accuracy: 0.7811

```
# K 탐색 및 10-Fold 교차 검증
k_values <- 1:200
results_class <- data.frame(k = integer(), cv_accuracy = numeric())

ctrl <- trainControl(method = "cv", number = 10)

for (k_val in k_values) {
  knn_model_class <- train(
    x = train_X,
    y = train_y_class,
    method = "knn",
    trControl = ctrl,
    tuneGrid = data.frame(k = k_val)
  )

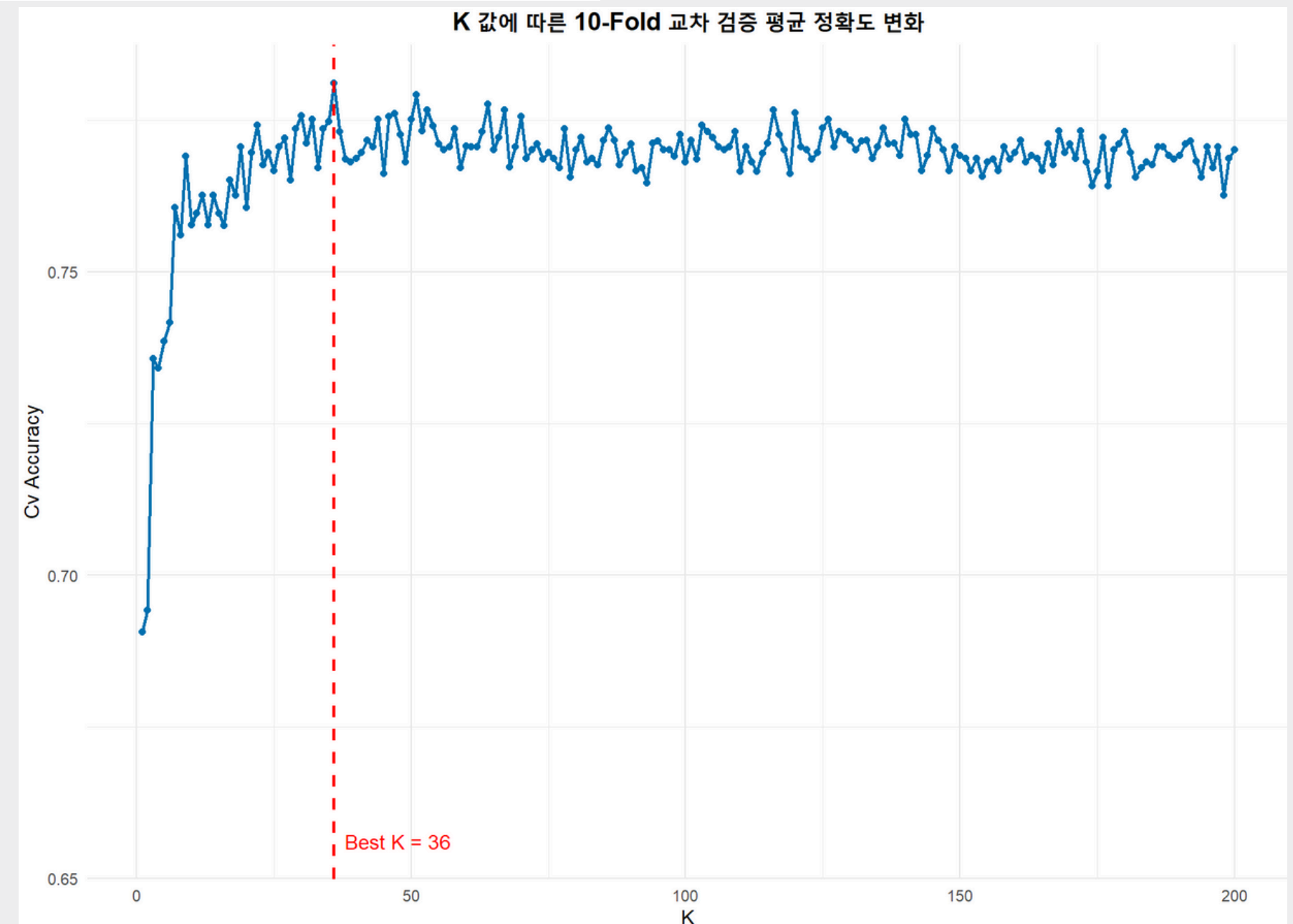
  mean_cv_accuracy <- max(knn_model_class$results$Accuracy)

  results_class <- rbind(results_class, data.frame(k = k_val, cv_accuracy = mean_cv_accuracy))
}

# 최적의 K 찾기
best_k_row <- results_class[which.max(results_class$cv_accuracy), ]
best_k <- best_k_row$k
best_cv_accuracy <- best_k_row$cv_accuracy

# 최종 정확도 계산
knn_predictions_best <- knn(
  train= train_X,
  test= test_X,
  cl= train_y_class,
  k= best_k
)
cm <- confusionMatrix(knn_predictions_best, test_y_class)
test_accuracy <- cm$overall['Accuracy']

# 결과 출력
cat("Best K 및 성능 지표\n")
cat("--- \n")
cat(sprintf("Best K: %d with CV Accuracy: %.4f\n", best_k, best_cv_accuracy))
cat("\n")
```



In the model of KNN, the optimal number is K=36, achieving the highest classification accuracy between 79% and 80%.

* Model Training

KNN

K (1~15)

Best K: 9 CV Accuracy: 0.7691

```
> print(results_class)
```

	k	cv_accuracy
1	1	0.6906572
2	2	0.6941686
3	3	0.7356730
4	4	0.7341641
5	5	0.7385530
6	6	0.7416458
7	7	0.7606134
8	8	0.7560644
9	9	0.7690695
10	10	0.7576583
11	11	0.7596606
12	12	0.7626322
13	13	0.7576558
14	14	0.7626147
15	15	0.7596033

K (1~50)

Best K: 36 CV Accuracy: 0.7811

```
> print(results_class)
```

35	35	0.7746920
36	36	0.7810848
37	37	0.7731198
38	38	0.7685873
39	39	0.7680573
40	40	0.7686373
41	41	0.7696171
42	42	0.7716347
43	43	0.7705723
44	44	0.7751295
45	45	0.7661181
46	46	0.7756147
47	47	0.7760747
48	48	0.7725809
49	49	0.7681131
50	50	0.7750907

K (1~200)

Best K: 36 CV Accuracy: 0.7811

```
> print(results_class)
```

185	185	0.7676271
186	186	0.7706073
187	187	0.7705834
188	188	0.7691258
189	189	0.7685945
190	190	0.7691209
191	191	0.7711122
192	192	0.7715896
193	193	0.7681518
194	194	0.7655871
195	195	0.7706234
196	196	0.7671495
197	197	0.7706145
198	198	0.7625884
199	199	0.7686332
200	200	0.7700734

* Model Training

Decision Tree

```
df$user_id <- NULL

df <- as.data.frame(df)

set.seed(123)
index <- createDataPartition(df$productivity_score, p = 0.8, list = FALSE)
train <- df[index, ]
test <- df[-index, ]

train <- as.data.frame(train)
test <- as.data.frame(test)

tree_model <- rpart(productivity_score ~ age + gender + physical_activity_hrs_pweek +
                    weight_kg + drink_pweek + quality_of_sleep + sleephrs +
                    stress_level + job_satisfaction + burnout_level,
                    data = train,
                    method = "anova")

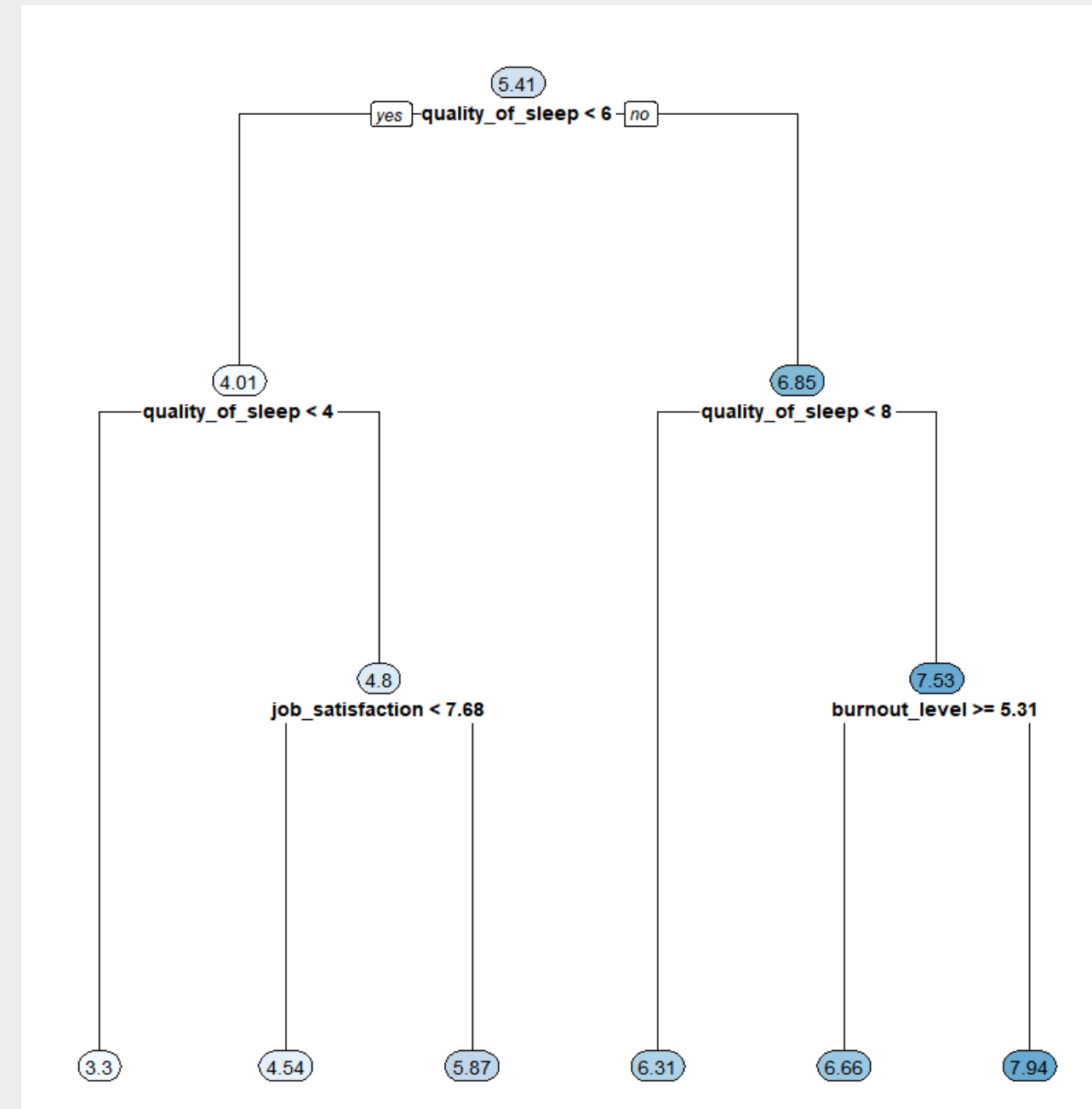
rpart.plot(tree_model,
           type = 2,
           fallen.leaves = TRUE,
           digits = 3,
           cex = 0.9,
           extra = 0)

tree_pred <- predict(tree_model, newdata = test)

tree_rmse <- sqrt(mean((tree_pred - test$productivity_score)^2))
tree_r2 <- cor(tree_pred, test$productivity_score)^2

tree_rmse
tree_r2
```

```
> tree_rmse
[1] 1.862675
> tree_r2
[1] 0.4346442
```



High sleep quality and low stress consistently led to the highest productivity, while poor sleep combined with high stress resulted in the largest decline.

* Model Training

Lasso Regression

```
set.seed(123)
lasso_cv <- cv.glmnet(x_train, y_train, alpha = 1)
plot(lasso_cv)

best_lambda <- lasso_cv$lambda.min
best_lambda
lasso_model <- glmnet(x_train, y_train, alpha = 1, lambda = best_lambda)
coef(lasso_model)

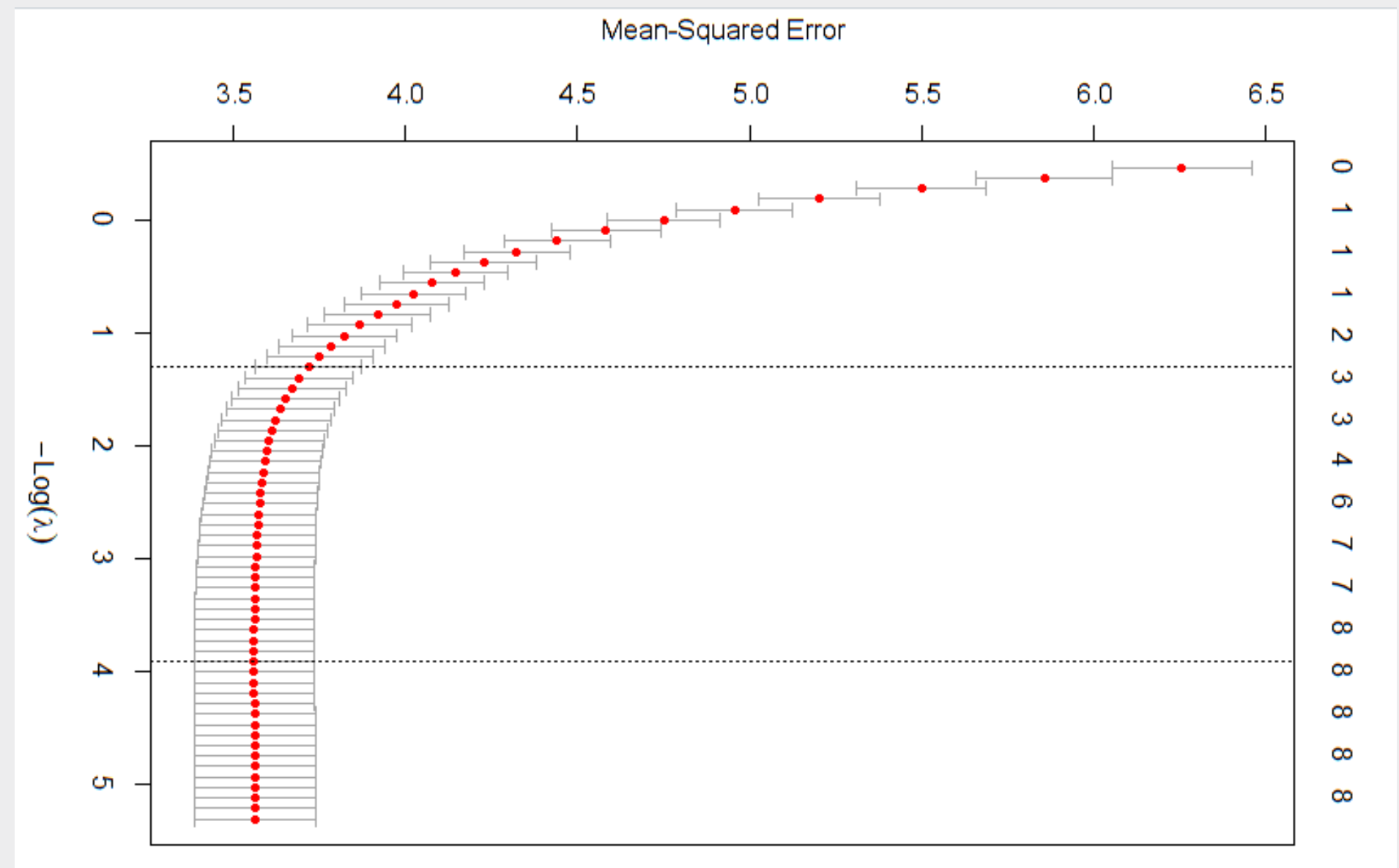
lasso_pred <- predict(lasso_model, s = best_lambda, newx = x_test)

lasso_rmse <- sqrt(mean((lasso_pred - y_test)^2))
lasso_r2 <- cor(lasso_pred, y_test)^2

lasso_rmse
lasso_r2
```

Sleep quality was the strongest positive predictor of productivity, while stress and burnout reduced it, and alcohol use and sleep duration had little effect.

```
> lasso_rmse
[1] 1.79752
> lasso_r2
           [,1]
s=0.02007214 0.4748908
```

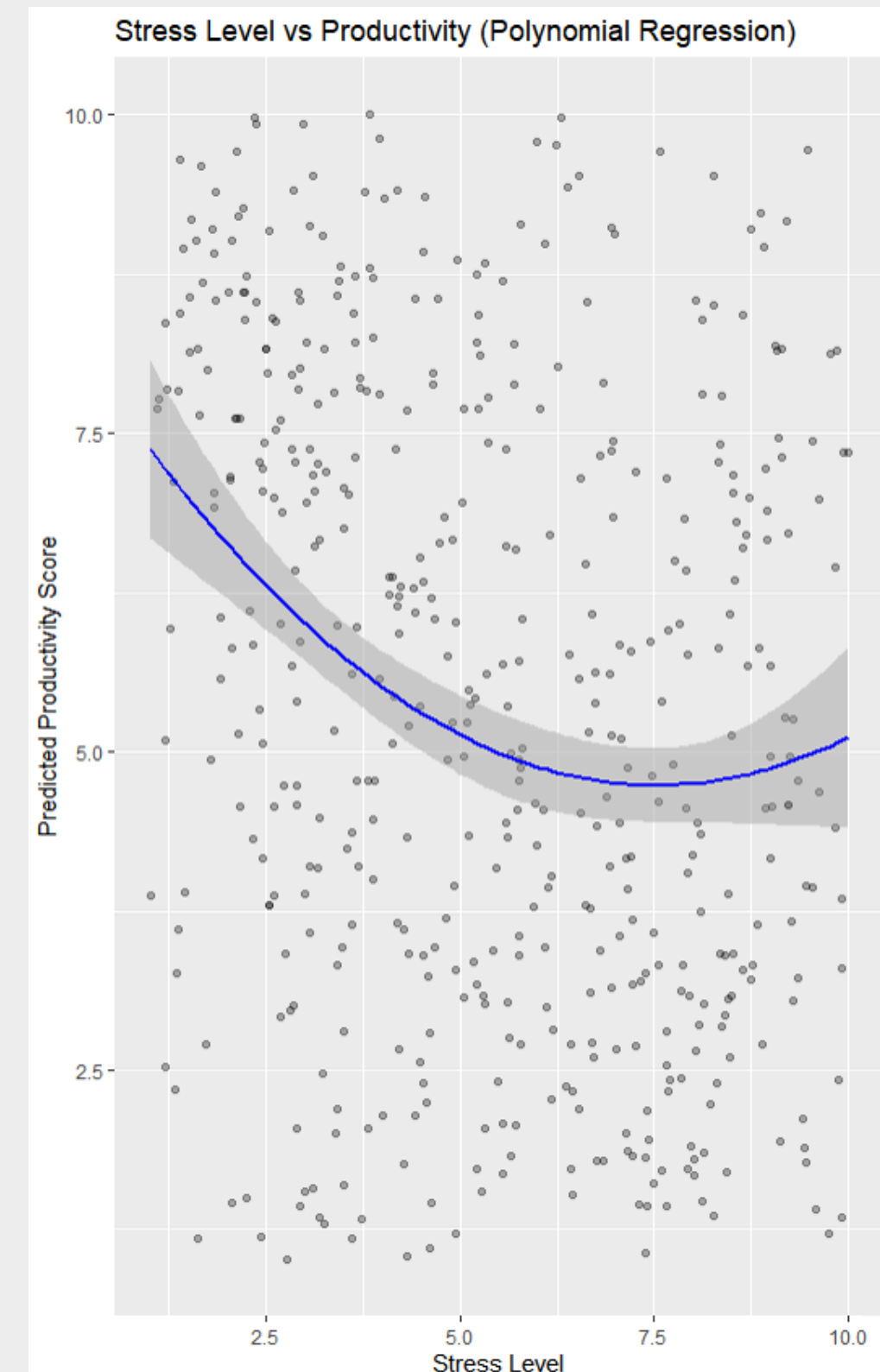


* Model Training

Polynomial Regression

```
poly_model <- lm(productivity_score ~  
  poly(stress_level, 2) +  
  poly(quality_of_sleep, 2) +  
  poly(job_satisfaction, 2) +  
  physical_activity_hrs_pweek +  
  burnout_level,  
  data = train)  
  
summary(poly_model)  
  
poly_pred <- predict(poly_model, test)  
  
poly_rmse <- sqrt(mean((poly_pred - test$productivity_score)^2))  
poly_r2 <- cor(poly_pred, test$productivity_score)^2  
  
poly_rmse  
poly_r2
```

A curved pattern emerged: moderate stress supported optimal productivity, whereas very low or very high stress reduced it.



```
> poly_rmse  
[1] 1.791016  
> poly_r2  
[1] 0.477539
```


* Model Performance Comparison

● Linear Regression	MSE: 1.86011
● RandomForest	MSE:1.8005
● KNN	Accuracy: 78% (K=36), classification model, MSE cannot be calculated.
● Decision Tree	MSE: 1.862675
● Lasso Regression	MSE: 1.79752
● Polynomial Regression	MSE: 1.791016

* Conclusion



**Polynomial
Regression**

The average square error of the Polynomial Regression model was the least, so it was selected as the optimal model.

The factor that most affects fatigue is selected as the highest amount of sleep in the importance of several models.

* Conclusion

Limitations

1. **Lack of variables and absence of context**

Lack of key contextual data such as breaks, caffeine intake, work intensity, and occupation

2. **Data Limits**

The data itself may have limitations in self-reporting data centered → reliability.

Improvements

1. Explore **more diverse data**, accurately measured data

2. Use **time-based modeling techniques** to reflect the process of change

3. Use **personalized forecasting methods** with individual data